



BD-141 Big Data

CUC — Cuatrimestre II, 2026

# Human Behavior AI

Detección de Manipulación Digital  
y Comportamiento Artificial

## Equipo 1:

Mariana Méndez Pérez · Luis Diego Montero Vargas · Josué Redondo Gómez · Claret Rodríguez Jiménez ·  
Nadin Rojas López

Prof. Ericka Valverde Navarro

# Agenda

---

**01** El Problema

**02** Objetivo del Proyecto

**03** Dataset y Procesamiento

**04** Arquitectura del Sistema

**05** Modelos de IA (Random Forest + Isolation Forest)

**06** Integración y Automatización

**07** Escalabilidad y Nube

**08** Riesgos, Costos y Mejoras

**09** Conclusiones y Lecciones Aprendidas

# El Problema

**\$100,000 millones USD**

en pérdidas anuales estimadas por fraude publicitario digital causado por bots

## Bots y cuentas automatizadas

Simulan ser humanos en redes sociales

## Métricas manipuladas

Likes, comentarios y tendencias falsas

## Opinión pública afectada

Campañas políticas y empresariales distorsionadas

## Detección compleja

Métodos tradicionales insuficientes para escala real

# Objetivo del Proyecto

---

## Objetivo General

Desarrollar una solución basada en Big Data e Inteligencia Artificial que permita identificar y analizar patrones de comportamiento artificial en plataformas digitales, mediante el procesamiento de datos y detección de anomalías en la actividad de los usuarios.

- 1 Recopilar y preparar datasets relacionados con comportamiento digital
- 2 Analizar patrones de interacción para identificar actividades sospechosas
- 3 Implementar modelos de IA para detección de comportamiento artificial
- 4 Desarrollar visualizaciones que faciliten la interpretación de anomalías
- 5 Construir un pipeline automatizado de detección en tiempo real

# Dataset Utilizado

**50,000**

Registros totales

**11**

Columnas originales

**30**

Columnas tras Fase 3

**50% / 50%**

Bots vs Humanos

## Variables originales

- User ID · Username · Tweet
- Retweet Count · Mention Count
- Follower Count · Verified
- Bot Label · Location
- Created At · Hashtags

## Variables creadas (Fase 3)

- engagement\_ratio · actividad\_total
- ratio\_retweet\_mencion
- franja\_horaria · antiguedad\_dias
- score\_sospecha · usa\_hashtags
- largo\_tweet · verificado\_int

# Arquitectura del Sistema

## 1 — INGESTA

Datasets Kaggle + Datos simulados



## 2 — ALMACENAMIENTO

SQLite · raw\_events · processed\_sessions



## 3 — PROCESAMIENTO

Python + Pandas · Limpieza · Variables



## 4 — IA

Scikit-Learn · Isolation Forest · Random Forest



## 5 — VISUALIZACIÓN

Streamlit · Dashboard · Alertas

# Procesamiento de Datos — Fase 3

## Limpieza realizada

- 8,341 nulos en Hashtags → "sin\_hashtags"
- Conversión de Created At a datetime64
- Extracción de año, mes, hora y día
- Estandarización de texto (strip + title)
- Eliminación de registros duplicados
- Verificado convertido a entero (0/1)

## Variables creadas

### Actividad

engagement\_ratio · actividad\_total · ratio\_retweet\_mencion

### Temporales

franja\_horaria · es\_fin\_de\_semana · antiguedad\_dias

### Contenido

cantidad\_hashtags · largo\_tweet · palabras\_tweet · usa\_hashtags

### Riesgo

alta\_actividad\_pocos\_seguidores · no\_verif\_alto\_engagement · activo\_madrugada · score\_sospecha

# Consultas SQL sobre la Base de Datos — Fase 3

---

Además de cargar los datos en SQLite, se ejecutaron consultas SQL reales sobre la tabla `processed_sessions`, demostrando lectura y análisis directo desde la base de datos.

## Consulta 1

### Conteo de cuentas por tipo

```
SELECT bot_label, COUNT(*) FROM processed_sessions GROUP BY  
bot_label
```

## Consulta 2

### Engagement promedio por tipo de cuenta

```
SELECT bot_label, AVG(engagement_ratio) FROM processed_sessions  
GROUP BY bot_label
```

## Consulta 3

### Top 5 cuentas con mayor score de sospecha

```
SELECT user_id, score_sospecha FROM processed_sessions ORDER BY  
score_sospecha DESC LIMIT 5
```

## Consulta 4

### Cuentas activas de madrugada, por tipo

```
SELECT bot_label, COUNT(*) FROM processed_sessions WHERE  
franja_horaria='madrugada' GROUP BY bot_label
```



# Reproducibilidad — Scripts Independientes

Además de los notebooks, el procesamiento está disponible como scripts de Python independientes en la carpeta /scripts del repositorio, ejecutables sin depender de Jupyter.

## 01\_carga.py

### Carga del dataset original y exploración inicial

**Corrección aplicada:** Ruta calculada automáticamente con `__file__`, funciona sin importar desde dónde se ejecute

## 02\_limpieza.py

### Limpieza de datos: nulos, duplicados y formatos

**Corrección aplicada:** Corrección de import con `importlib` para permitir ejecución standalone

## 03\_procesamiento.py

### Creación de 19 variables y carga a SQLite

**Corrección aplicada:** Corrección de import con `importlib` para permitir ejecución standalone

Los 3 scripts corren de forma encadenada: carga -> limpieza -> procesamiento -> carga a SQLite, sin necesidad de abrir un notebook.

# Análisis Exploratorio — Fase 4

---

## Antigüedad de cuenta

A simple vista, las cuentas bot parecían tener cuentas más nuevas, pero la diferencia real entre promedios fue de apenas 0.13%.

## Engagement ratio

El engagement ratio parecía distinto por unos pocos valores extremos, pero al filtrarlos la diferencia real fue de solo - 6.26%.

## Conteo de seguidores

La distribución de seguidores es prácticamente idéntica entre bots y humanos, ambas uniformes entre 0 y 10,000.

## Franja horaria

La actividad por franja horaria, incluida la madrugada, resultó prácticamente igual entre cuentas bot y humanas.

*Nota: las diferencias visuales entre bots y humanos existían, pero no fueron suficientes para que el modelo las aprovechara de forma efectiva.*

# Modelo de IA — Random Forest

## Configuración

|              |                                |
|--------------|--------------------------------|
| Algoritmo:   | Random Forest (supervisado)    |
| Árboles:     | 100 estimadores                |
| Profundidad: | max_depth = 10                 |
| Pesos:       | class_weight = balanced        |
| Features:    | 16 variables de comportamiento |
| División:    | 80% entrenamiento / 20% prueba |

## Métricas obtenidas

|            |        |
|------------|--------|
| Accuracy:  | 49.54% |
| Precision: | 49.59% |
| Recall:    | 51.14% |
| F1-Score:  | 50.35% |

## Top 3 variables más importantes para la detección

**1°** antigüedad\_dias  
Importancia: 0.1344

**2°** Follower Count  
Importancia: 0.1309

**3°** engagement\_ratio  
Importancia: 0.1184

# Hallazgo Técnico — Resultado del Modelo

## Un Accuracy del ~50% equivale a clasificación aleatoria

Esto no es un fallo del modelo — es un hallazgo técnico relevante sobre el dataset

### Dataset sin variables de sesión

No incluye tiempos entre eventos, secuencias de clics ni patrones de navegación, que son las variables más discriminativas para detectar bots.

### Etiquetas poco discriminativas

Las variables disponibles no presentan separación estadística suficiente entre bots y humanos para que ningún modelo pueda aprender.

### Confirmado con dos modelos

Random Forest (supervisado) e Isolation Forest (no supervisado) obtuvieron resultados idénticos (~50%), lo que confirma que el problema está en los datos.

# Isolation Forest — Modelo Complementario

## ¿Por qué Isolation Forest?

Modelo no supervisado que detecta anomalías sin necesitar etiquetas previas. No depende de la calidad del Bot Label, por lo que puede encontrar patrones que Random Forest no detectó.

| Métrica   | Random Forest | Isolation Forest |
|-----------|---------------|------------------|
| Accuracy  | 49.54%        | 49.40%           |
| Precision | 49.59%        | 49.44%           |
| Recall    | 51.14%        | 49.56%           |
| F1-Score  | 50.35%        | 49.50%           |
| Tipo      | Supervisado   | No supervisado   |

**Conclusión:** ambos modelos confirmaron el mismo hallazgo. El problema está en las variables del dataset, no en el algoritmo utilizado.

# Integración y Automatización — Fase 5

Kaggle

**50,000**

registros históricos

% bots: 50.0%

Twitter  
simulado

**2,000**

registros generados

% bots: 44.9%

Reddit  
simulado

**1,500**

registros generados

% bots: 31.9%

**Total integrado: 53,500 registros desde 3 fuentes**

5 lotes × 200 registros procesados en tiempo real

Tiempo de proceso por lote: < 0.05 segundos

Pipeline automatizado end-to-end con 7 etapas

Sistema de alertas con umbral de probabilidad 0.80

# Pipeline Automatizado



# Procesamiento Síncrono — Fase 5

---

El pipeline implementado es de tipo síncrono: cada etapa espera a que la anterior termine, ya que depende directamente de su resultado.

## ACTUAL — Síncrono

Cada etapa (validación, limpieza, clasificación, alertas, reporte) se ejecuta en orden y espera a que la anterior termine.

Adecuado para el volumen actual: lotes de 200 registros procesados en menos de 0.05 segundos.

## FUTURO — Asíncrono

Con mayor volumen, varias etapas podrían ocurrir al mismo tiempo, sin esperar unas a otras, usando colas de mensajes.

Tecnología propuesta: Apache Kafka, contemplada en la estrategia de escalabilidad de la Fase 6.

## ¿Por qué elegir síncrono ahora?

Porque cada etapa depende del resultado de la anterior (no se puede clasificar un registro que no se ha limpiado), y el volumen actual no justifica la complejidad adicional de un procesamiento asíncrono.



# Escalabilidad del Sistema — Fase 6

## Etapa Actual (Académico)

~53,500 registros

Python + Pandas  
SQLite local

## Escala Media (Startup)

Millones de registros

PostgreSQL particionado  
Apache Spark

## Escala Alta (Producción)

Miles de millones

Spark + Kafka  
Cloud Storage

### Estrategias de escalabilidad implementadas:

Particionado horizontal por fecha en PostgreSQL

Réplicas de lectura para el dashboard

Procesamiento asíncrono con colas de mensajes

Migración progresiva a Apache Spark

# Arquitectura Cloud y Seguridad

## Google Cloud Platform

|                 |                        |
|-----------------|------------------------|
| Ingesta:        | Cloud Pub/Sub          |
| Procesamiento:  | Dataflow (Apache Beam) |
| Almacenamiento: | BigQuery + Cloud SQL   |
| Modelo IA:      | Vertex AI              |
| Visualización:  | Looker Studio          |
| Orquestación:   | Cloud Composer         |

Costo estimado: ~\$164 USD/mes (con Cloud Scheduler)

## Seguridad

### Cifrado:

TLS 1.3 en tránsito  
AES-256 en reposo

### Acceso:

IAM + principio de  
mínimo privilegio

### Datos:

Anonimización de  
user\_id con hash

### Normas:

GDPR · CCPA  
Ley 8968 CR

# Riesgos del Sistema y Costos

| Nivel | Riesgo                    | Estrategia de mitigación                           |
|-------|---------------------------|----------------------------------------------------|
| Alto  | Bots que evaden el modelo | Reentrenamiento periódico con datos recientes      |
| Alto  | Degradación del modelo    | Monitoreo continuo de métricas y alertas           |
| Medio | Dataset desbalanceado     | Técnicas SMOTE + class_weight balanceado           |
| Medio | Falsos positivos          | Umbral ajustable + revisión manual de casos límite |
| Alto  | Cambios en APIs externas  | Conectores intercambiables y fuentes alternativas  |

|                                       |                |                |                 |                 |                                |               |
|---------------------------------------|----------------|----------------|-----------------|-----------------|--------------------------------|---------------|
| Estimación de costos mensuales en GCP |                |                |                 |                 | Total estimado: ~\$164 USD/mes |               |
| Pub/Sub: \$8                          | Dataflow: \$35 | BigQuery: \$60 | Vertex AI: \$18 | Cloud Run: \$15 | Scheduler: \$5                 | Storage: \$23 |

# Mejoras Futuras y Optimización

## Corto plazo (1-2 semanas)

- Ajuste de hiperparámetros con GridSearchCV
- Optimización del umbral de clasificación
- Validación cruzada más robusta

## Mediano plazo (1-2 meses)

- Incorporar variables de sesión y secuencia
- Implementar XGBoost como modelo alternativo
- Exponer modelo como API REST con FastAPI
- Dashboard en tiempo real con Looker Studio

## Largo plazo (3-6 meses)

- Migración completa a GCP
- Kafka + Spark Streaming en tiempo real
- Modelos LSTM para análisis de secuencias
- Reentrenamiento automático mensual
- Sistema de alertas por Slack o correo

# Conclusiones

---

## Sistema funcional completo

Se desarrolló una solución end-to-end que integra múltiples fuentes, procesa datos en tiempo real y clasifica cuentas automáticamente.

## Hallazgo técnico honesto

El modelo obtuvo ~50% de accuracy, revelando que el dataset no cuenta con variables suficientemente discriminativas para detectar bots de forma efectiva.

## Arquitectura sólida

La arquitectura por capas permite escalar progresivamente desde el entorno académico hasta producción en GCP sin rediseñar la solución.

## Seguridad y gobernanza

El sistema contempla cifrado, control de acceso, anonimización y cumplimiento con normativas GDPR, CCPA y Ley 8968 CR.

## Camino claro hacia adelante

La incorporación de variables de sesión y secuencia es el paso más importante para superar el rendimiento actual del modelo.

# Lecciones Aprendidas

---

**Mariana  
Méndez**

La calidad del dataset es más importante que la complejidad del modelo. Sin buenas variables, ningún algoritmo puede compensar la falta de información.

**Luis Diego  
Montero**

Diseñar la arquitectura por capas desde el inicio facilita enormemente el trabajo colaborativo y la detección de errores en cada etapa.

**Josué  
Redondo**

Los resultados honestos tienen más valor académico que los resultados inflados. Documentar el porqué de un bajo rendimiento es un hallazgo técnico legítimo.

**Claret  
Rodríguez**

La automatización de pipelines ahorra tiempo en fases posteriores. Lo que parece complejo al principio se vuelve reutilizable en cada iteración.

**Nadín  
Rojas**

La escalabilidad debe planificarse desde el diseño, no agregarse al final. Las decisiones de arquitectura tempranas definen los límites del sistema a futuro.

# Resumen del Proyecto

---

## Fase 1

Definición del problema

## Fase 2

Arquitectura y modelado

## Fase 3

Procesamiento de datos

## Fase 4

Análisis e IA

## Fase 5

Integración y automatización

## Fase 6

Escalabilidad, nube y optimización

6 fases completadas · 5 notebooks · 22 visualizaciones · 2 modelos de IA · 1 pipeline automatizado

# Evidencia Funcional Disponible

Todo el trabajo del proyecto está documentado, versionado y disponible públicamente para su revisión.

6

## Notebooks

Uno por fase (Fase 3 a Fase 6),  
cada uno ejecutable de principio  
a fin sin errores

3

## Scripts .py

Carga, limpieza y procesamiento,  
ejecutables de forma  
independiente

1

## Dashboard

Streamlit con 6 secciones:  
resumen, análisis, modelo,  
pipeline, nube y dataset  
explorable

22

## Visualizaciones

Gráficos generados a partir de  
datos reales del proyecto, no  
imágenes genéricas

**Repositorio:** [github.com/Clachar94/HumanBehaviorAI](https://github.com/Clachar94/HumanBehaviorAI)

Rama principal actualizada, con historial completo de commits de las 6 fases del proyecto.



# Human Behavior AI

Detección de Manipulación Digital y Comportamiento Artificial

BD-141 Big Data · CUC · Cuatrimestre II, 2026

## ¿Preguntas?

**Equipo 1:**

Mariana Méndez Pérez · Luis Diego Montero Vargas · Josué Redondo Gómez · Claret Rodríguez Jiménez · Nadin Rojas López

Prof. Ericka Valverde Navarro